

# Advanced Programming

## Module introduction

Ayoub FRIHAOUI

`ayoub.frihaoui@lecnam.net`

Conservatoire national des arts et métiers (CNAM Paris)

# Where you are

You already know how to write Java that **works**: classes, inheritance, collections.

This module is about Java that **lasts**:

- easy to **read** six months later;
- easy to **change** without breaking everything;
- easy to **test** without a real database or a real file.

## One-sentence pitch

From “I can make it run” to “I can design it so others (and future you) can keep it running”.

# Why this module exists

The code you will meet at work is not a student `Main`. It is Spring, Hibernate/JPA, JUnit, Guice, Lombok. . .

## What looks like magic

- `@Test` methods run themselves
- `@Entity` becomes a SQL table
- `@Inject` fills a field
- a typed list stays type-safe

## What it actually is

- introspection and annotations
- object-relational mapping
- inversion of control
- generics

If you understand these ideas, frameworks stop being magic. You can **debug** them, **interview** on them, and **choose** a design instead of copying a tutorial.

# What you will learn

- ➊ **Metaprogramming** — inspect and annotate code at runtime (the machinery behind JUnit, JPA, Spring).
- ➋ **Generics** — write type-safe APIs without casts everywhere.
- ➌ **Object design** — less coupling, clearer responsibilities, code you can change.
- ➍ **Design patterns** — reusable solutions to recurring design problems.
- ➎ **Dependency injection** — make dependencies explicit, so the code is testable.

Lectures introduce the ideas. Labs make you use them.

# How we work

- **Lectures** and **labs**: do not skip the lecture before the lab.
- Each lab is a Maven project. The gate is the tests, run with the **Maven wrapper** shipped in the lab (no need to install Maven): `mvnw.cmd test` on Windows.
- Use **Git** from the start. Small commits.
- Read the tests: they are the specification.
- Prefer a simple design you can explain over a clever one you cannot.

## About AI tools

You may use them as a tutor (“explain this error”). You may *not* submit code you cannot explain.

# What you need on your machine

## Install

- JDK 25
- Git
- IntelliJ IDEA or VS Code

## Do not install

Maven is already in each lab  
(`mvnw` / `mvnw.cmd`).

Before the first lab, check:

- `java -version` → 25
- `git -version`

Then, inside a lab folder:

- `mvnw.cmd test`

## To be determined

Weights, deliverables, exam format and late-work rules will be announced later. Do not assume a scheme yet.

What you can already count on:

- labs matter — keep them compiling and the tests green;
- you must be able to explain your own code.

Questions: [ayoub.frihaoui@lecnam.net](mailto:ayoub.frihaoui@lecnam.net)